Token-based authentication provides a solution to this problem by allowing you to generate a long-lived token that you can give to the third-party service instead of your password. The service can then use the token to act on your behalf. When you stop using the service you can revoke the token to prevent any further access.

Though using a token means that you don't need to give the third-party your password, the tokens you've used so far still grant full access to APIs as if you were performing actions yourself. The third-party service can use the token to do anything that you can do. But you may not trust a third-party to have full access, and only want to grant them partial access. When I ran my own business, I briefly used a third-party service to read transactions from my business bank account and import them into the accounting software I used. Although that service needed only read access to recent transactions, in practice it had full access to my account and could have transferred funds, cancelled payments, and performed many other actions. I stopped using the service and went back to manually entering transactions because the risk was too great.[1]

The solution to these issues is to restrict the API operations that can be performed with a token, allowing it to be used only within a well-defined *scope*. For example, you might let your accounting software read transactions that have occurred within the last 30 days, but not let it view or create new payments on the account. The scope of the access you've granted to the accounting software is therefore limited to read-only access to recent transactions. Typically, the scope of a token is represented as one or more string